Đặt vé đơn hàng và hóa đơn

Tài liệu dành cho người phụ trách luồng chọn phim, giữ ghế, tạo đơn và phát hành hóa đơn.

Một đơn có thể gồm vé, bắp nước hoặc cả hai. Trình duyệt chỉ giữ lựa chọn tạm; ghế được giữ khi máy chủ tạo đơn thành công. Sau khi xác nhận thanh toán, hệ thống mới phát hành hóa đơn tương ứng với phần vé và phần bắp nước.

Quy ước đường dẫn: UI/... là src/main/resources/UI/...; lớp Java nằm dưới src/main/java/com/cinesnack/ theo controller, service, repository, dto hoặc entity. Mỗi đoạn code trích dẫn có đường dẫn đầy đủ và số dòng để bạn tìm lại trong IDE.

1 Bản đồ luồng

1. **Chọn phim:** movies.html/movies.js hoặc booking.html/booking.js → GET /api/movies — CatalogController → CatalogService → CatalogRepository đọc PHIM
2. **Chọn suất và ghế:** booking.js → GET /api/showtimes?movieId=... → GET /api/showtimes/{id}/seats — CatalogRepository đọc SUAT_CHIEU, GHE và VE để đánh dấu ghế còn trống
3. **Giữ giỏ tạm:** booking.js → sessionStorage → cinema.js → checkout.js — chưa ghi SQL; gioHopLe() chỉ xác minh dữ liệu trình duyệt
4. **Tạo đơn:** checkout.js → OrderController/GuestOrderController/AdminOrderController — CreateOrderRequest nhận JSON, @Valid kiểm tra các trường
5. **Chốt đơn:** OrderService → BookingCatalogRepository → OrderRepository — đọc lại suất/ghế/giá, khóa suất, tính tổng và ghi DON_HANG, VE, chi tiết bắp nước
6. **Xem hoặc hủy:** orders.js/admin.js → controller → OrderService → OrderRepository — đọc chi tiết và chỉ hủy đơn CHO_THANH_TOAN
7. **Thanh toán rồi phát hành:** PaymentService → InvoiceService → InvoiceRepository — ghi HOA_DON_VE và/hoặc HOA_DON_BONG_NUOC
8. **Xem hóa đơn:** orders.js → InvoiceController hoặc GuestOrderController → InvoiceService → InvoiceRepository — trả về hóa đơn có các dòng hàng đã chụp tại lúc mua

2 Vai trò từng file

File	Việc file thực hiện và hàm cần đọc
UI/html/movies.html + movies.js	Hiển thị phim và nút đi tới trang đặt vé; movies.js gọi GET /api/movies.
UI/html/booking.html	Các vùng chọn phim, ngày, suất, ghế và bắp nước; nạp booking.js.
UI/assets/js/booking.js	doiPhim() tải suất; hienGio() hiện giờ; chonSuat() tải ghế; lưu giỏ vào sessionStorage.
UI/assets/js/cinema.js	gioHopLe() kiểm tra giỏ; tinhTien() tính tạm; taoYeuCau() đổi giỏ thành JSON của API.
UI/assets/js/booking-summary.js	hienTomTat() hiển thị phim, ghế và giá trên hai trang.
UI/html/checkout.html + checkout.js	Lấy email, chọn API tạo đơn theo vai trò, gửi yêu cầu và chuyển sang trang xem đơn.
CatalogController/Service/Repository	Cung cấp danh sách phim, suất và sơ đồ ghế công khai; đọc PHIM, SUAT_CHIEU, GHE, VE.
CreateOrderRequest.java	Định nghĩa email, showTimeId, seatIds, snacks, combos; ràng buộc @Email,

	@Size, @Positive.
OrderController.java	API tạo, xem, liệt kê, hủy đơn của tài khoản đăng nhập.
GuestOrderController.java	API khách vãng lai; GuestOrderAccess lưu mã đơn được tạo vào HttpSession.
AdminOrderController.java	POST /api/admin/orders tạo tại quầy; GET liệt kê đơn toàn rạp cho nhân viên/quản trị.
OrderService.java	Kiểm tra lựa chọn, suất, ghế, mặt hàng; tính tổng; kiểm tra quyền và trạng thái khi xem/hủy.
BookingCatalogRepository.java	Khóa suất bằng UPDLOCK/HOLDLOCK và đọc lại ghế, giá, trạng thái từ SQL.
OrderRepository.java	Ghi DON_HANG/VE/chi tiết bắp nước; đọc chi tiết; đổi trạng thái; tìm đơn hết hạn.
OrderExpiryService.java	Mỗi 30 giây tìm đơn quá thời hạn giữ chỗ và chuyển sang HUY.
Order.java/Ticket.java/OrderItem.java	Các record dữ liệu trả về JSON đơn, vé và dòng bắp nước.
UI/html/orders.html + orders.js	Hiện danh sách, chi tiết, QR và hóa đơn; cho hủy đơn đang chờ.
InvoiceController.java	GET /api/orders/{code}/invoices cho tài khoản đăng nhập.
InvoiceService.java	Phát hành cùng giao dịch thanh toán; kiểm tra quyền trước khi đọc hóa đơn.
InvoiceRepository.java	Tạo hóa đơn vé/bắp nước riêng; lưu ảnh chụp tên/giá tại lúc mua, rồi đọc dòng hóa đơn.
database/cinesnack.sql	Bảng PHIM, SUAT_CHIEU, GHE, DON_HANG, VE, CHI_TIET_HOA_DON_BONG_NUOC, HOA_DON_VE, HOA_DON_BONG_NUOC.

3 Đọc các đoạn lệnh quan trọng

Mã nguồn: src/main/resources/UI/assets/js/booking.js · dòng 136–157

```
136    async function doiPhim(maUuTien = null) {
137      const lan = ++lanTai;
138      g.phim = phim.find((p) => p.id === Number(o("chon-phim").value)) || null;
139      g.ghe = [];
140      g.suat = null;
141      g.maSuatChieu = null;
142      dangTai = true;
143      capNhat();
144      o("chon-ngay").replaceChildren();
145      o("chon-gio").replaceChildren();
146      o("hang-ghe").replaceChildren();
147      o("ghe-loi").textContent = "";
148      hienTrangThaiGhe(g.phim ? "Đang tải suất chiếu…" :
149        o("chon-phim").value === "snacks" ? "Chỉ mua bắp nước: không cần chọn ghế." : "Chọn phim trước để
xem suất chiếu.");
150      loi("");
151      if (!g.phim) {
152        dangTai = false;
153        capNhat();
154        return;
155      }
156      try {
157        const ds = await CineSnackApi.goi("/api/showtimes?movieId=" + g.phim.id);
```

Giải thích: doiPhim() lấy id phim được chọn, xóa ghế và suất cũ để tránh dùng nhầm dữ liệu, rồi gọi /api/showtimes?movieId=... . await đợi phản hồi trước khi dựng lựa chọn ngày, giờ.

Mã nguồn: src/main/resources/UI/assets/js/booking.js · dòng 48–67

```
48    async function chonSuat(s) {
49      const lan = ++lanTai;
50      g.suat = s;
51      g.maSuatChieu = s?.id || null;
52      g.ghe = [];
53      ghe = [];
54      o("ghe-loi").textContent = "";
55      hienTrangThaiGhe(s ? "Đang tải sơ đồ ghế…" : "Chọn suất chiếu để xem ghế.");
56      dangTai = true;
57      capNhat();
58      o("hang-ghe").replaceChildren();
59      if (!s) {
60        dangTai = false;
61        capNhat();
62        return;
63      }
64      try {
65        const ds = await CineSnackApi.goi(`/api/showtimes/${s.id}/seats`);
66        if (lan !== lanTai) return;
67        ghe = ds;
```

Giải thích: chonSuat() ghi suất mới vào giỏ, bỏ danh sách ghế cũ, gọi API ghế của suất đó. Biến lanTai giúp bỏ qua phản hồi mạng đến muộn sau khi người dùng đã đổi suất.

Mã nguồn: src/main/resources/UI/assets/js/booking.js · dòng 245–254

```
245    o("chon-phim").addEventListener("change", () => doiPhim());
246    o("tiep-dat-ve").addEventListener("click", () => {
247      if (!CineSnack.gioHopLe(g)) {
248        loi("Suất hoặc lựa chọn không còn hợp lệ. Hãy chọn lại.");
249        return;
250      }
251      try {
252        sessionStorage.setItem(CineSnack.khoaGio, JSON.stringify(g));
253        location.href = "checkout.html";
254      } catch {
```

Giải thích: Sự kiện click kiểm tra giỏ, JSON.stringify đổi đối tượng thành chuỗi và sessionStorage giữ tạm cho checkout.html. Đây chưa phải hành động giữ ghế trong SQL.

Mã nguồn: src/main/resources/UI/assets/js/cinema.js · dòng 108–121

```
108    function taoYeuCau(g, email) {
109      return {
110        email,
111        showTimeId: g.maSuatChieu,
112        seatIds: g.ghe.map((v) => v.id),
113        snacks: g.snacks.map((v) => ({
114          productId: v.productId,
115          quantity: v.quantity,
116        })),
117        combos: g.combos.map((v) => ({
118          comboId: v.comboId,
119          quantity: v.quantity,
120        })),
121      };
```

Giải thích: taoYeuCau() chỉ gửi email, showTimeId, seatIds, productId/comboId và quantity; không gửi giá. Máy chủ tự xác định giá để không bị sửa từ trình duyệt.

Mã nguồn: src/main/resources/UI/assets/js/checkout.js · dòng 53–61

```
53      gui = true;
54      nut.disabled = true;
55      nut.setAttribute("aria-busy", "true");
56      loi.textContent = "Đang tạo đơn…";
57      try {
58        const staff = user?.role === "NHAN_VIEN";
59        const d = await CineSnackApi.goi(staff ? "/api/admin/orders" : user ? "/api/orders" :
"/api/guest/orders", {
60          method: "POST",
61          body: CineSnack.taoYeuCau(g, email.value.trim()),
```

Giải thích: Biểu thức ba nhánh chọn API: staff → /api/admin/orders; user → /api/orders; guest → /api/guest/orders. Cùng một DTO giúp ba luồng dùng chung logic OrderService.

Mã nguồn: src/main/java/com/cinesnack/dto/CreateOrderRequest.java · dòng 9–17

```
     @JsonProperty("email") @NotBlank @Email @Size(max = 150) String thuDienTu,
     @JsonProperty("showTimeId") @Positive Long maSuatChieu,
     @JsonProperty("seatIds") @NotNull @Size(max = 10) List<@NotNull @Positive Long> dsMaGhe,
     @JsonProperty("snacks") @NotNull @Size(max = 20) List<@NotNull @Valid SnackChoice> dsBapNuoc,
     @JsonProperty("combos") @NotNull @Size(max = 20) List<@NotNull @Valid ComboChoice> dsCombo
 ) {
     public record SnackChoice(
         @JsonProperty("productId") @NotNull @Positive Long maSanPham,
         @JsonProperty("quantity") @Min(1) @Max(10) int soLuong
```

Giải thích: record là kiểu dữ liệu gọn của Java; @JsonProperty ánh xạ tên trường JSON tiếng Anh sang tên biến Java; @NotNull/@Size/@Positive chặn dữ liệu sai ở cửa API.

Mã nguồn: src/main/java/com/cinesnack/service/OrderService.java · dòng 68–93

```
     }

     private Order taoDonHang(CreateOrderRequest yeuCau, Long maNguoiDung) {
         kiemTraLuaChon(yeuCau);
         var suatChieu = yeuCau.dsMaGhe().isEmpty()
             ? null
             : danhMucDatVe.timVaKhoaSuatChieu(yeuCau.maSuatChieu());
         LocalDateTime hienTai = LocalDateTime.now(dongHo).truncatedTo(ChronoUnit.SECONDS);
         LocalDateTime hetHan = hienTai.plusMinutes(15);
         BigDecimal giaVe = BigDecimal.ZERO;

         if (!yeuCau.dsMaGhe().isEmpty()) {
             if (
                 suatChieu == null ||
                 !"SAP_CHIEU".equals(suatChieu.trangThai()) ||
                 !suatChieu.batDau().isAfter(hienTai)
             ) {
                 throw loiXungDot("Suất chiếu không còn nhận đặt vé");
             }
             if (suatChieu.batDau().isBefore(hetHan)) hetHan = suatChieu.batDau();
             giaVe = suatChieu.gia();
             for (long maGhe : yeuCau.dsMaGhe()) {
                 var ghe = danhMucDatVe.timGhe(maGhe);
                 if (
                     ghe == null ||
                     ghe.maPhong() != suatChieu.maPhong() ||
```

Giải thích: @Transactional bao cả quá trình tạo đơn. Hệ thống đọc lại suất với khóa, xác minh thời gian chiếu, phòng và ghế. Thời hạn giữ chỗ tối đa 15 phút, hoặc dừng ở lúc suất bắt đầu nếu sớm hơn.

Mã nguồn: src/main/java/com/cinesnack/service/OrderService.java · dòng 112–152

```
                 )
             );
         }
         for (var luaChon : yeuCau.dsCombo()) {
             dsMatHang.add(
                 taoDongBapNuoc(
                     null,
                     luaChon.maCombo(),
                     danhMucDatVe.timCombo(luaChon.maCombo()),
                     luaChon.soLuong()
                 )
             );
         }
         BigDecimal tongTien = giaVe.multiply(BigDecimal.valueOf(yeuCau.dsMaGhe().size()));
         for (var dongChiTiet : dsMatHang)
             tongTien = tongTien.add(
                 dongChiTiet.matHang().gia().multiply(BigDecimal.valueOf(dongChiTiet.soLuong()))
             );
         if (tongTien.compareTo(new BigDecimal("9999999999.99")) > 0) throw loiDuLieu(
             "Tổng tiền vượt giới hạn"
         );

```

```
        String maDonHang = Codes.taoMa("DH");
        long maSo = khoDonHang.taoDonHang(
            maNguoiDung,
            maDonHang,
            yeuCau.thuDienTu(),
            tongTien,
            hetHan,
            hienTai
        );
        for (long maGhe : yeuCau.dsMaGhe())
            khoDonHang.themVe(maSo, yeuCau.maSuatChieu(), maGhe, giaVe);
        for (var dongChiTiet : dsMatHang)
            khoDonHang.themBapNuoc(
                maSo,
                dongChiTiet.maSanPham(),
                dongChiTiet.maCombo(),
                dongChiTiet.matHang().ten(),
                dongChiTiet.soLuong(),
                dongChiTiet.matHang().gia()
```

Giải thích: Máy chủ tính tổng bằng BigDecimal từ giá trong cơ sở dữ liệu, sinh mã DH, ghi đơn rồi ghi từng vé và dòng bắp nước. Nếu một thao tác lỗi, transaction hoàn tác toàn bộ.

Mã nguồn: src/main/java/com/cinesnack/repository/OrderRepository.java · dòng 113–148

```
    public boolean gheDaDuocDat(long maSuat, long maGhe) {
        Integer n = truyVan.queryForObject(
            "SELECT COUNT(*) FROM VE WHERE suat_chieu_id=? AND ghe_id=? AND trang_thai IN
('CHO_THANH_TOAN','DA_THANH_TOAN','DA_SOAT')",
            Integer.class,
            maSuat,
            maGhe
        );
        return n != null && n > 0;
    }

    public void themVe(long maDon, long maSuat, long maGhe, BigDecimal gia) {
        int n = truyVan.update(
            """
            INSERT INTO VE(don_hang_id,suat_chieu_id,ghe_id,phong_id,ma_ve,don_gia,trang_thai)
            SELECT ?,s.suat_chieu_id,?,s.phong_id,?,?,'CHO_THANH_TOAN'
            FROM SUAT_CHIEU s WHERE s.suat_chieu_id=?
            """,
            maDon,
            maGhe,
            Codes.taoMa("VE"),
            gia,
            maSuat
        );
        if (n != 1) throw new IllegalStateException("Thiếu suất chiếu");
    }

    public void themBapNuoc(
        long maDon,
        Long maSanPham,
        Long maCombo,
        String ten,
        int soLuong,
        BigDecimal gia
    ) {
        int n = truyVan.update(
            """
```

Giải thích: INSERT ... OUTPUT INSERTED.don_hang_id tạo đơn và lấy khóa mới; INSERT VE gắn don_hang_id với suất/ghế. Dấu ? là chỗ truyền tham số bằng JdbcTemplate.

Mã nguồn: src/main/java/com/cinesnack/service/OrderExpiryService.java · dòng 27–44

```
    @Scheduled(fixedDelay = 30000, initialDelay = 60000)
    public void xuLyHetHan() {
        LocalDateTime hienTai = LocalDateTime.now(dongHo).truncatedTo(ChronoUnit.SECONDS);
        for (String ma : khoDonHang.layMaDonHetHan(hienTai)) {
            // Mỗi đơn có transaction riêng; cùng khóa với thanh toán và hủy đơn.
            giaoDich.executeWithoutResult(trangThai -> {
```

```
33                  var donHang = khoDonHang.timVaKhoa(ma);
34                  if (donHang != null &&
35                          "CHO_THANH_TOAN".equals(donHang.trangThai()) &&
36                          donHang.hetHanLuc() != null &&
37                          !donHang.hetHanLuc().isAfter(hienTai) &&
38                          khoDonHang.doiTrangThai(donHang.maSo(), "CHO_THANH_TOAN", "HUY") == 1) {
39                      khoDonHang.dongChiTiet(donHang.maSo(), "HUY");
40                  }
41              });
42          }
43      }
44  }
```

Giải thích: @Scheduled chạy lặp 30 giây sau lần chờ đầu 60 giây. Đơn còn CHO_THANH_TOAN và hết thời gian được đổi sang HUY, đồng thời chi tiết không còn giữ ghế.

Mã nguồn: src/main/java/com/cinesnack/service/InvoiceService.java · dòng 25–30

```
25      @Transactional(propagation = Propagation.MANDATORY)
26      public void phatHanh(Order donHang, LocalDateTime hienTai) {
27          khoHoaDon.taoHoaDon(donHang, hienTai);
28      }
29
30      @Transactional(readOnly = true)
```

Giải thích: Propagation.MANDATORY buộc phát hành hóa đơn nằm trong giao dịch thanh toán đang mở. Vì thế không tạo hóa đơn riêng khi đơn chưa được xác nhận đã trả tiền.

Mã nguồn: src/main/java/com/cinesnack/repository/InvoiceRepository.java · dòng 58–80

```
58              truyVan.update("UPDATE VE SET hoa_don_ve_id=? WHERE don_hang_id=?", id, donHang.maSo());
59          }
60          if (tienBap != null) {
61              truyVan.update(
62                      "INSERT INTO
HOA_DON_BONG_NUOC(don_hang_id,ma_hoa_don,tong_tien,trang_thai,email_nhan,tao_luc,phat_hanh_luc)
VALUES(?,?,?,'DA_THANH_TOAN',?,?,?)",
63                      donHang.maSo(), Codes.taoMa("HDB"), tienBap, donHang.thuDienTu(), luc, luc);
64              long id = truyVan.queryForObject("SELECT hoa_don_bong_nuoc_id FROM HOA_DON_BONG_NUOC WHERE
don_hang_id=?",
65                      Long.class, donHang.maSo());
66              truyVan.update("UPDATE CHI_TIET_HOA_DON_BONG_NUOC SET hoa_don_bong_nuoc_id=? WHERE
don_hang_id=?", id,
67                      donHang.maSo());
68          }
69          if (tienVe == null && tienBap == null)
70              throw new IllegalStateException("Đơn không có chi tiết");
71      }
72
73      public List<Invoice> timTheoDonHang(long maDon) {
74          var ds = new ArrayList<Invoice>();
75          for (String loai : List.of("VE", "BONG_NUOC")) {
76              // Tên bảng/cột lấy từ tập cố định nội bộ, không nhận từ request.
77              String bang = "VE".equals(loai) ? "HOA_DON_VE" : "HOA_DON_BONG_NUOC";
78              String cot = "VE".equals(loai) ? "hoa_don_ve_id" : "hoa_don_bong_nuoc_id";
79              var headers = truyVan.query(
80                      "SELECT * FROM " + bang + " WHERE don_hang_id=? AND trang_thai='DA_THANH_TOAN'",
```

Giải thích: SUM(don_gia) cộng tiền vé; SUM(so_luong*don_gia) cộng bắp nước. Chỉ phần nào có chi tiết mới tạo hóa đơn phần đó và liên kết ngược tới VE/chi tiết bắp nước.

4 Ví dụ một đơn gồm hai loại hàng

Khách chọn 2 ghế, một sản phẩm bắp và một combo. checkout.js chỉ gửi các id và số lượng. OrderService đọc lại giá, tạo một DON_HANG và các dòng VE, CHI_TIET_HOA_DON_BONG_NUOC. Khi nhân viên xác nhận thanh toán, PaymentService đổi đơn sang DA_THANH_TOAN; InvoiceRepository phát hành một HOA_DON_VE và một HOA_DON_BONG_NUOC. orders.js gọi API hóa đơn rồi hiển thị hai loại riêng.

5 Quy tắc và trường hợp lỗi

- **Ghế bị tranh mua:** Repository kiểm tra vé đã giữ/bán và ràng buộc SQL ngăn một ghế bị bán hai lần; API trả xung đột để người dùng chọn lại.
- **Chỉ mua bắp nước:** Không chọn suất/ghế vẫn hợp lệ nếu có ít nhất một sản phẩm hoặc combo.
- **Giỏ trống:** OrderService.kiemTraLuaChon() từ chối đơn không có vé lẫn bắp nước.
- **Thời hạn:** Ngày hết hạn là giới hạn giữ chỗ; khi quá hạn, trạng thái đơn là HUY, không phải HET_HAN.
- **Hủy:** Chỉ đơn CHO_THANH_TOAN được hủy; đơn DA_THANH_TOAN không đi qua luồng hủy này.
- **Hóa đơn:** Chỉ phát hành sau thanh toán thành công; một đơn có thể có một hoặc hai hóa đơn tùy loại hàng.
- **Giá hiển thị:** Giá trong giỏ là thông tin tạm; số tiền chính thức được tính từ SQL khi tạo đơn.

6 Cách lần theo một yêu cầu tạo đơn

JSON ví dụ: email = khach@example.com, showTimeId = 12, seatIds = [31, 32], snacks = [{productId: 5, quantity: 2}], combos = []. CreateOrderRequest nhận đúng các tên trường này. OrderService kiểm tra suất 12 còn tương lai, ghế 31/32 thuộc phòng và chưa bị giữ, sản phẩm 5 còn bán; sau đó dùng giá thật trong SQL để tính tổng. Khi repository ghi xong, API trả mã đơn và trạng thái CHO_THANH_TOAN. Nếu đã có người giữ ghế 31, transaction thất bại và không để lại nửa đơn.

Muốn kiểm tra hóa đơn, bắt đầu ở PaymentService.thanhToanTaiQuay() hoặc xacNhanChuyenKhoan(), tìm lệnh phatHanh(), rồi đọc InvoiceRepository.taoHoaDon(). Khi mở đơn trên giao diện, theo orders.js → GET /api/orders/{code}/invoices → InvoiceController → InvoiceService → InvoiceRepository.timTheoDonHang().

7 Từ vựng kỹ thuật

- **sessionStorage:** Bộ nhớ tạm trong tab trình duyệt; không phải cơ sở dữ liệu và không khóa ghế.
- **@Valid:** Yêu cầu Spring kiểm tra các ràng buộc ghi trong DTO trước khi xử lý nghiệp vụ.
- **UPDLOCK, HOLDLOCK:** Khóa của SQL Server để giảm khả năng hai yêu cầu đồng thời cùng chốt một suất/ghế.
- **@Transactional:** Đảm bảo các lần ghi đơn và dòng chi tiết thành một công việc thống nhất.
- **record:** Cú pháp Java ngắn cho đối tượng dữ liệu; mã nguồn dùng để biểu diễn request và response.